

APLICAÇÃO DE APRENDIZADO DE MÁQUINA E EQUILÍBRIO DE NASH NO JOGO DE PÔQUER

Autor: Gabriel Jooji Yamashiro

Orientador: Prof. Dr. Leandro Passos Junior

Banca: Profa . Dra. Simone das Graças Domingues Prado | Prof. Dr. Douglas Rodrigues

Universidade Estadual Paulista “Júlio de Mesquita Filho” (UNESP) Faculdade de Ciências (FC) /
Departamento de Computação (DCo) Bauru, SP - Brasil

Sumário

Introdução - (3)

Objetivo - (4)

Referencial Teórico - Como jogar - (4)

Kuhn Poker - CFR - (5)

Abstração - (6)

Metodologia - (7- 12)

Resultados (13-20)

Análise Geral - (21)

Conclusão (22)

- Teoria dos Jogos
- Equilíbrio de Nash
- Aplicação: Inteligência Artificial (IA), economia
- e negociações automatizadas ([RUSSELL; NORVIG, 2013](#))
- Diferença entre o pôquer e jogos como xadrez e Go.

- Pôquer exige técnicas avançadas para lidar com probabilidades, blefes e adaptação.
- Trabalhos como **Libratus** e **Pluribus** mostraram o potencial do algoritmo *Counterfactual Regret Minimization* (CFR)
- Desafio: soluções acessíveis em hardware modesto
- Como criar um agente IA que tome decisões próximas ao Equilíbrio de Nash?

Objetivo Geral

- Desenvolver um agente de inteligência artificial baseado em aprendizado de máquina que utilize o Equilíbrio de Nash para otimizar decisões no pôquer *Texas Hold'em Heads-Up*.

Objetivos Específicos

- Implementar um modelo de ML para identificar padrões de jogo e ajustar estratégias automaticamente;
- Aplicar algoritmos de Teoria dos Jogos para encontrar estratégias otimizadas baseadas no Equilíbrio de Nash;
- Avaliar a performance do agente em simulações contra adversários computacionais.

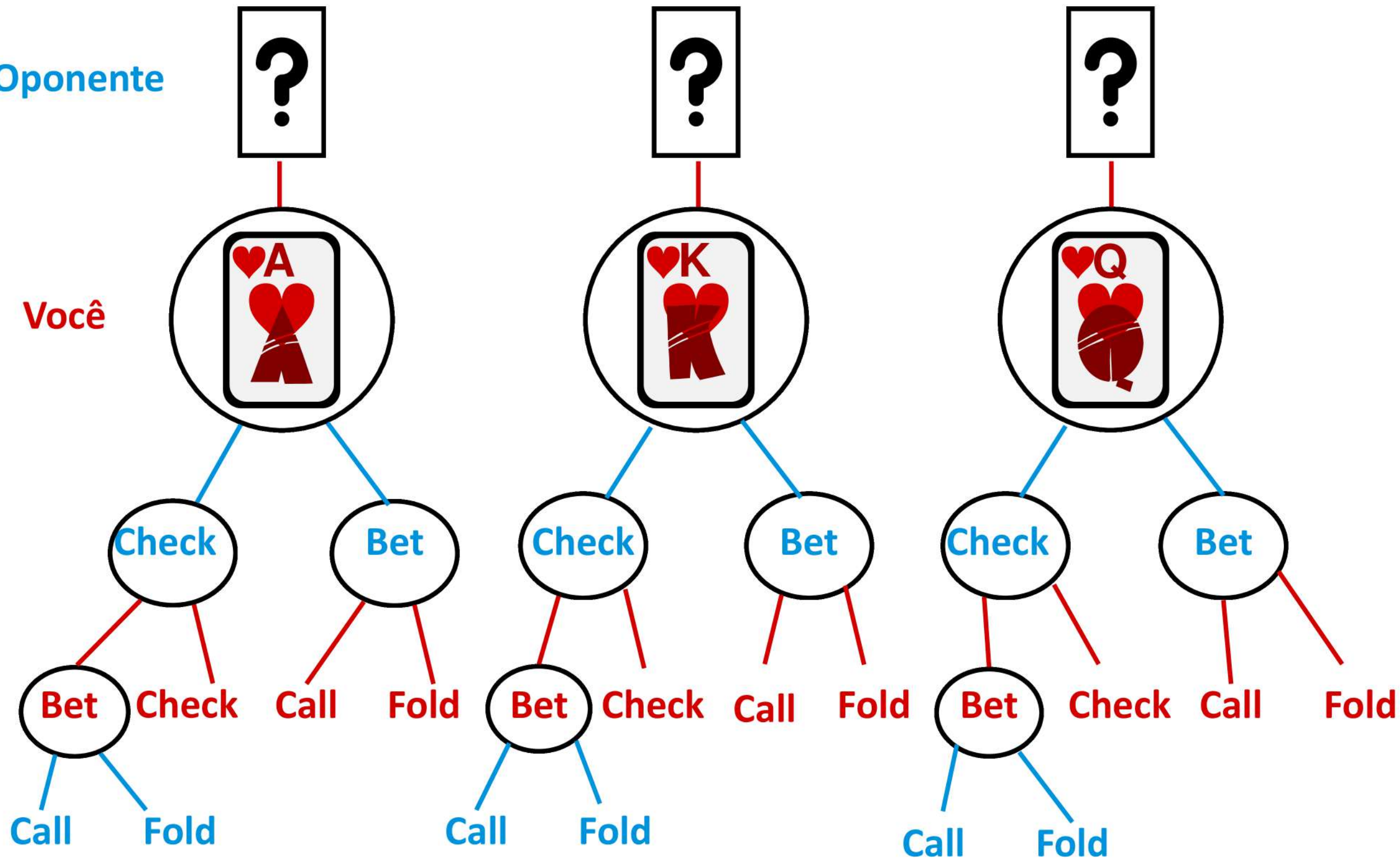


Fonte: <https://www.gipsyteam.com/poker/how-to-remember-poker-hands>

- Cada jogador recebe duas cartas privadas e será revelado até cinco cartas comunitárias
- **Flop**: três cartas abertas simultaneamente;
- **Turn**: uma quarta carta comunitária;
- **River**: a quinta e última carta comunitária.
- O jogo é dividido em quatro rodadas de apostas, nas quais os jogadores podem escolher entre as ações **fold** (desistir ou fugir), **check** (mesa ou passar a vez), **call** (pagar a aposta ou igualar a aposta), **bet/raise** (apostar ou aumentar a aposta) ou **all-in** (apostar todas as fichas).

Oponente

Você



Uma carta para cada jogador e apenas uma rodada de aposta

$Q < K < A$

Exemplo: Temos Q e o oponente aposta

Call (-2) e Fold (-1)

Abstração de ações

- Agrupar tamanhos de apostas parecidas (40-60% do pote) para ½ pote.

Abstração de cartas

- Abstração de Naipes
- Abstração de equidade



AA 85	AKs 67	AQs 66	AJs 65	ATs 65	A9s 63	A8s 62	A7s 61	A6s 60	A5s 60	A4s 59	A3s 58	A2s 57
AKo 65	KK 82	KQs 63	KJs 63	KTs 62	K9s 60	K8s 58	K7s 57	K6s 57	K5s 56	K4s 55	K3s 54	K2s 53
AQo 64	KQo 61	QQ 80	QJs 60	QTs 59	Q9s 58	Q8s 56	Q7s 54	Q6s 54	Q5s 53	Q4s 52	Q3s 51	Q2s 50
AJo 64	KJo 61	QJo 58	JJ 77	JTs 57	J9s 56	J8s 54	J7s 52	J6s 50	J5s 50	J4s 49	J3s 48	J2s 47
ATo 63	KTo 60	QTo 57	JTo 55	TT 75	T9s 54	T8s 52	T7s 51	T6s 49	T5s 47	T4s 46	T3s 46	T2s 45
A9o 61	K9o 58	Q9o 55	J9o 53	T9o 51	99 72	98s 51	97s 49	96s 47	95s 46	94s 44	93s 43	92s 42
A8o 60	K8o 56	Q8o 54	J8o 51	T8o 50	98o 48	88 69	87s 48	86s 46	85s 44	84s 43	83s 41	82s 40
A7o 59	K7o 55	Q7o 52	J7o 50	T7o 48	97o 46	87o 45	77 66	76s 45	75s 44	74s 42	73s 40	72s 38
A6o 58	K6o 54	Q6o 51	J6o 48	T6o 46	96o 44	86o 43	76o 42	66 63	65s 43	64s 41	63s 39	62s 38
A5o 58	K5o 53	Q5o 50	J5o 47	T5o 44	95o 43	85o 41	75o 40	65o 40	55 60	54s 41	53s 40	52s 38
A4o 57	K4o 52	Q4o 49	J4o 46	T4o 43	94o 41	84o 39	74o 38	64o 40	54o 38	44 57	43s 39	42s 37
A3o 56	K3o 51	Q3o 48	J3o 45	T3o 42	93o 40	83o 37	73o 37	63o 36	53o 36	43o 35	33 54	32s 36
A2o 55	K2o 50	Q2o 47	J2o 44	T2o 42	92o 39	82o 37	72o 35	62o 34	52o 34	42o 33	32o 32	22 50

Metodologia- Ambiente de Desenvolvimento

Os experimentos foram conduzidos em um ambiente controlado, utilizando um computador com as seguintes especificações técnicas:

- Processador: 13th Gen Intel® Core™ i5-13400 (2,50 GHz);
- Memória RAM: 32,0 GB;
- Placa de vídeo: 9 GB VRAM;
- Armazenamento: 1,82 TB SSD.

Para garantir a reprodutibilidade e o isolamento das dependências, foi configurado um ambiente virtual **Conda**, permitindo o controle preciso das versões das bibliotecas e a execução dos experimentos sem interferência de pacotes externos

Metodologia- Ferramentas utilizadas

O projeto foi desenvolvido integralmente em **Python 3.9**

As principais bibliotecas utilizadas foram:

- **NumPy**: utilizada para operações matemáticas vetorizadas e manipulação de matrizes de arrendimento e estratégias;
- **Scikit-learn (K-means)**: aplicada para a realização da abstração de estados, agrupando mãos semelhantes e reduzindo a dimensionalidade do espaço de decisão;
- **Joblib**: responsável pelo armazenamento e recuperação eficiente de dados intermediários, como batches de treinamento e estratégias parciais.

Foi implementada uma *Application Programming Interface*(API) de comunicação com o agente **Slumbot**.

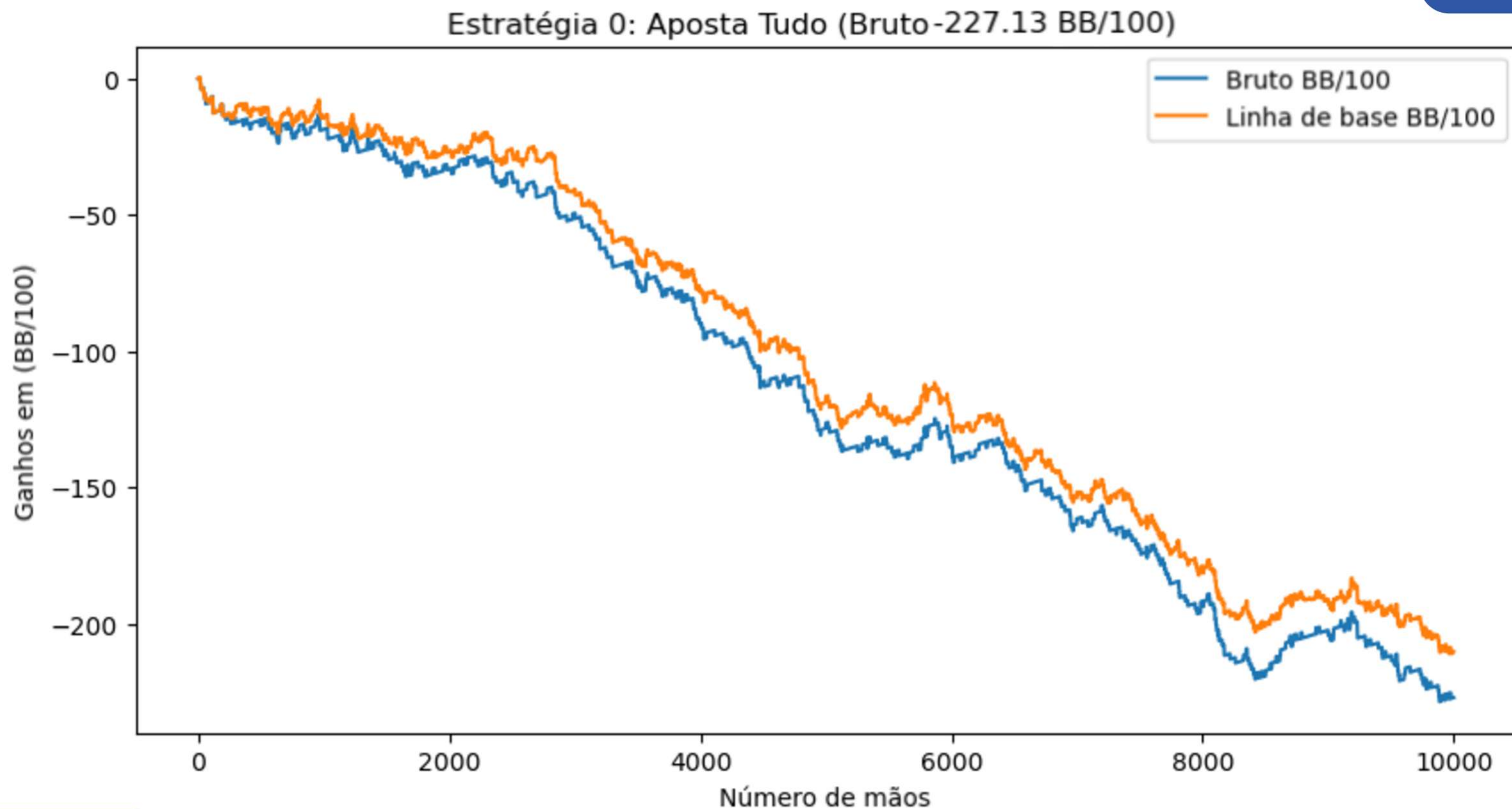
- Foi gerado um conjunto de dados contendo 1.000.000 mãos simuladas, representando diferentes cenários de jogo heads-up. Cada mão foi iterada diversas vezes até que o algoritmo CFR convergisse para o equilíbrio.
- Os resultados foram armazenados em batches de 50.000 mãos, o que permitiu um melhor controle da memória e uma análise incremental do aprendizado do agente

O agente CFR foi treinado utilizando a versão vanilla (versão simples do algoritmo), com atualização iterativa dos arrependimentos e das estratégias médias acumuladas. Durante o treinamento, o algoritmo alternou entre as seguintes etapas:

1. Cálculo do arrependimento contrafactual de cada ação;
2. Atualização das estratégias com base nos arrependimentos positivos acumulados.
3. Normalização e armazenamento das estratégias médias para posterior avaliação.

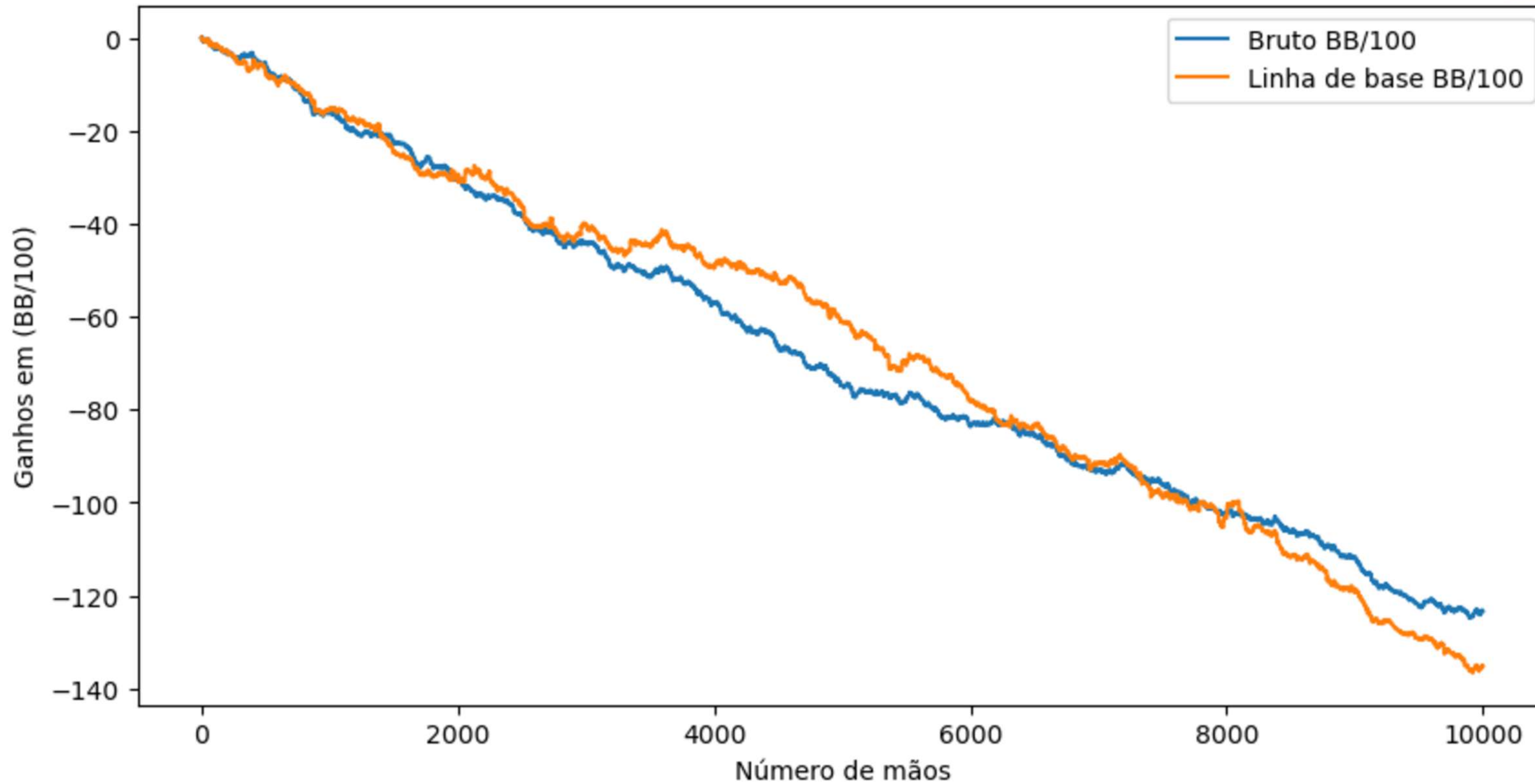
1. Configuração do ambiente **Conda** e inicialização do agente;
2. Geração do ***dataset*** com 1.000.000 de mãos;
3. Treinamento iterativo com batches de 50.000 (*pré-flop*)(totalizando 20 batches);
4. Treinamento iterativo com batches de 50.000 (*pós-flop*)(totalizando 20 batches);
5. Armazenamento intermediário via **Joblib**;
6. Avaliação contra heurísticas e **Slumbot**;
7. Coleta dos resultados e geração dos gráficos de desempenho.

Resultados - Heurísticas- Estratégia Aposta Tudo



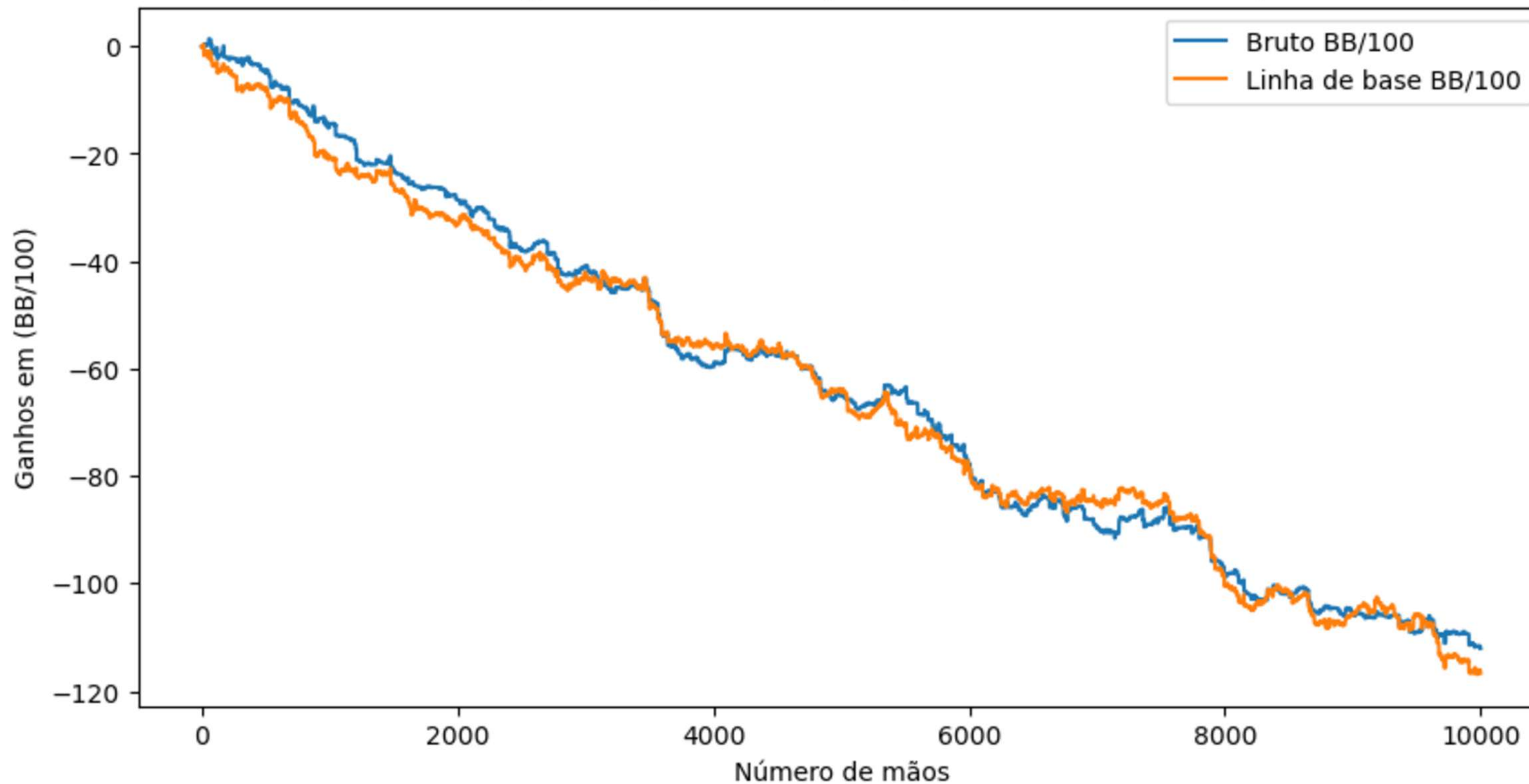
Resultados - Heurísticas- Estratégia Mesa ou paga

Estratégia 1: Mesa ou paga (Bruto -123.335 BB/100)



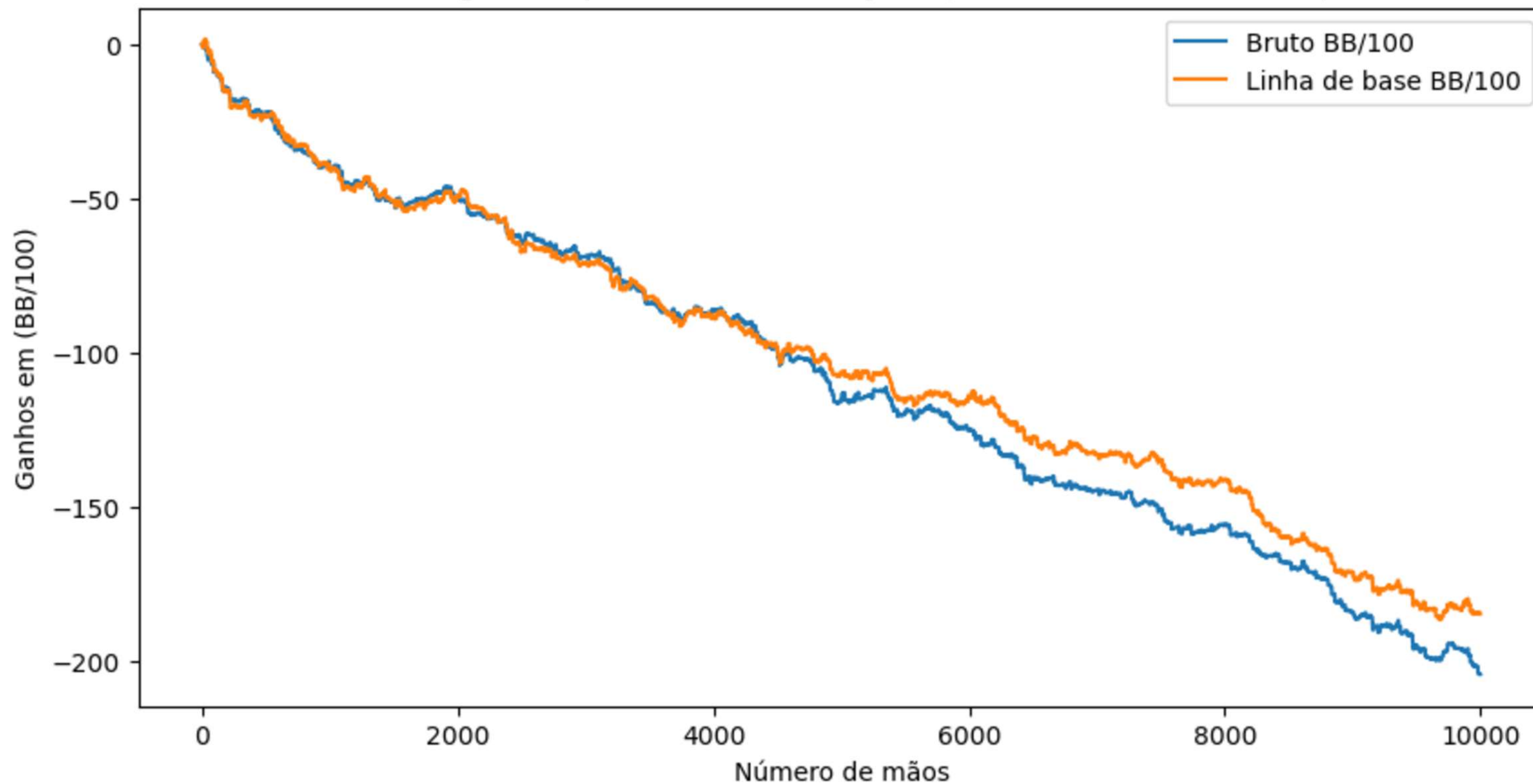
Resultados - Heurísticas - Estratégia Equidade Simples

Estratégia 2: Equidade Simples (Bruto -112.045 BB/100)

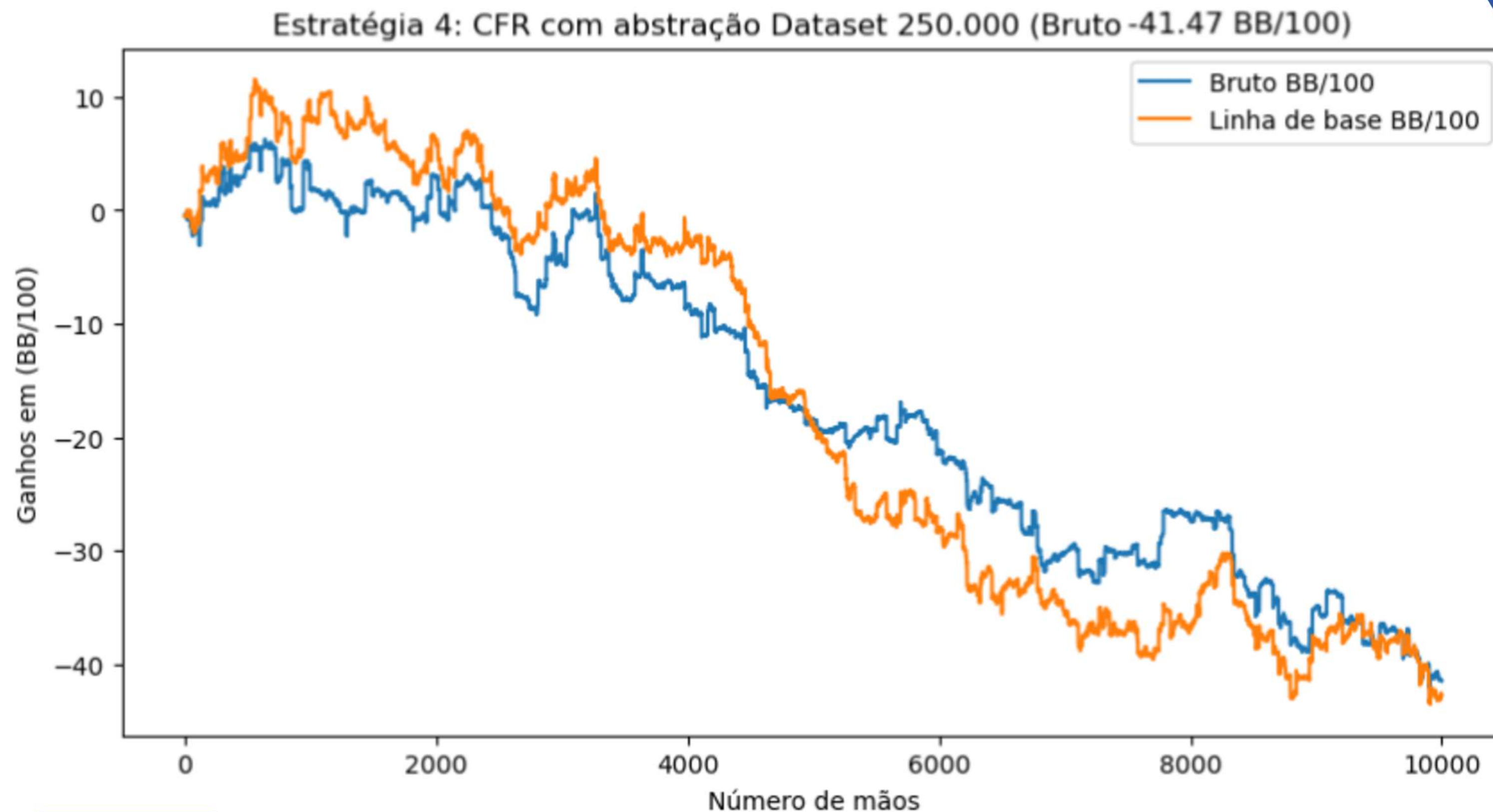


Resultados - Heurísticas- Estratégia Equidade mais avançada

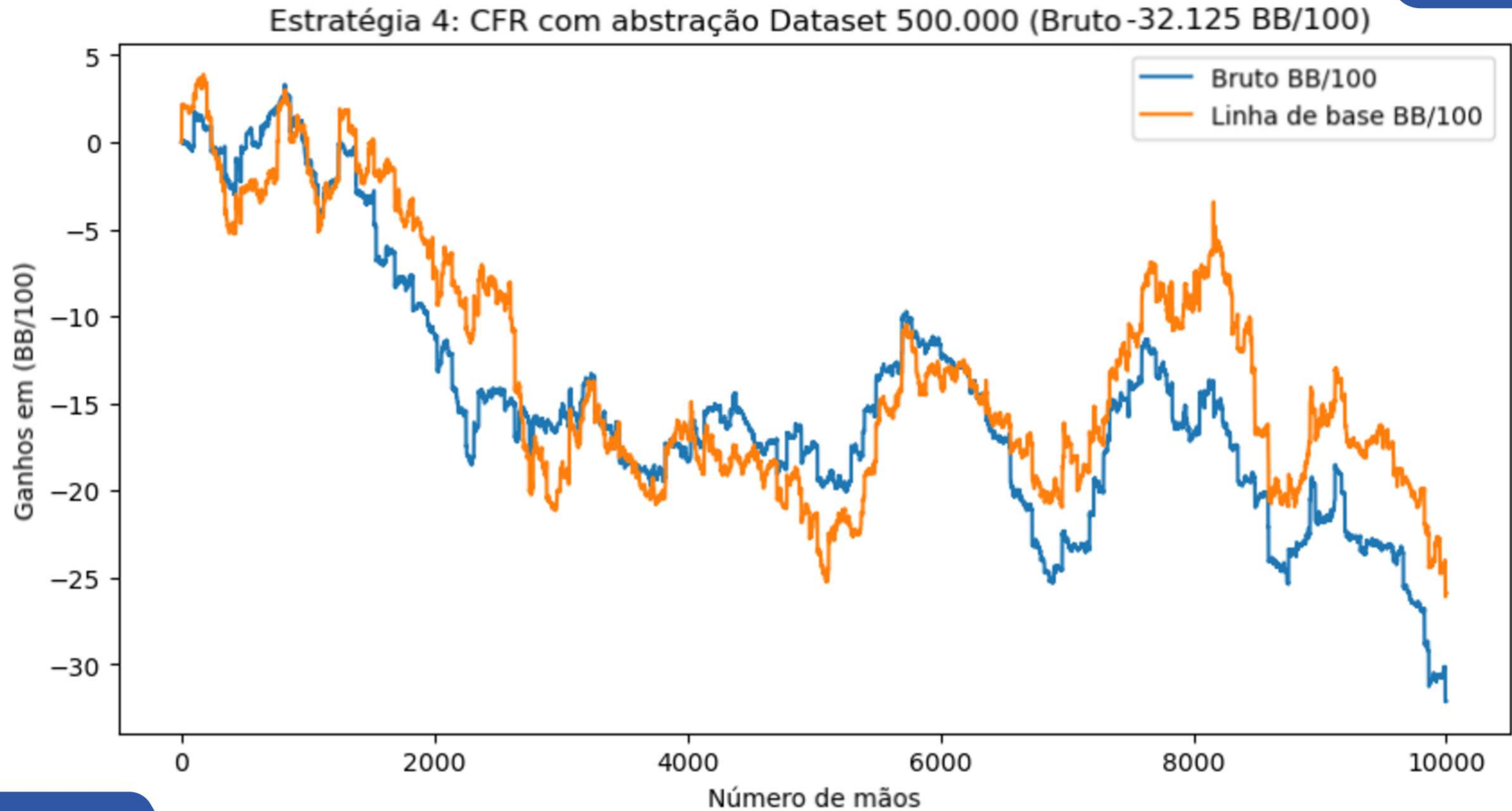
Estratégia 3: Equidade mais avançada (Bruto -204.2917 BB/100)



Resultados - CFR com Abstração-Dataset 250.000

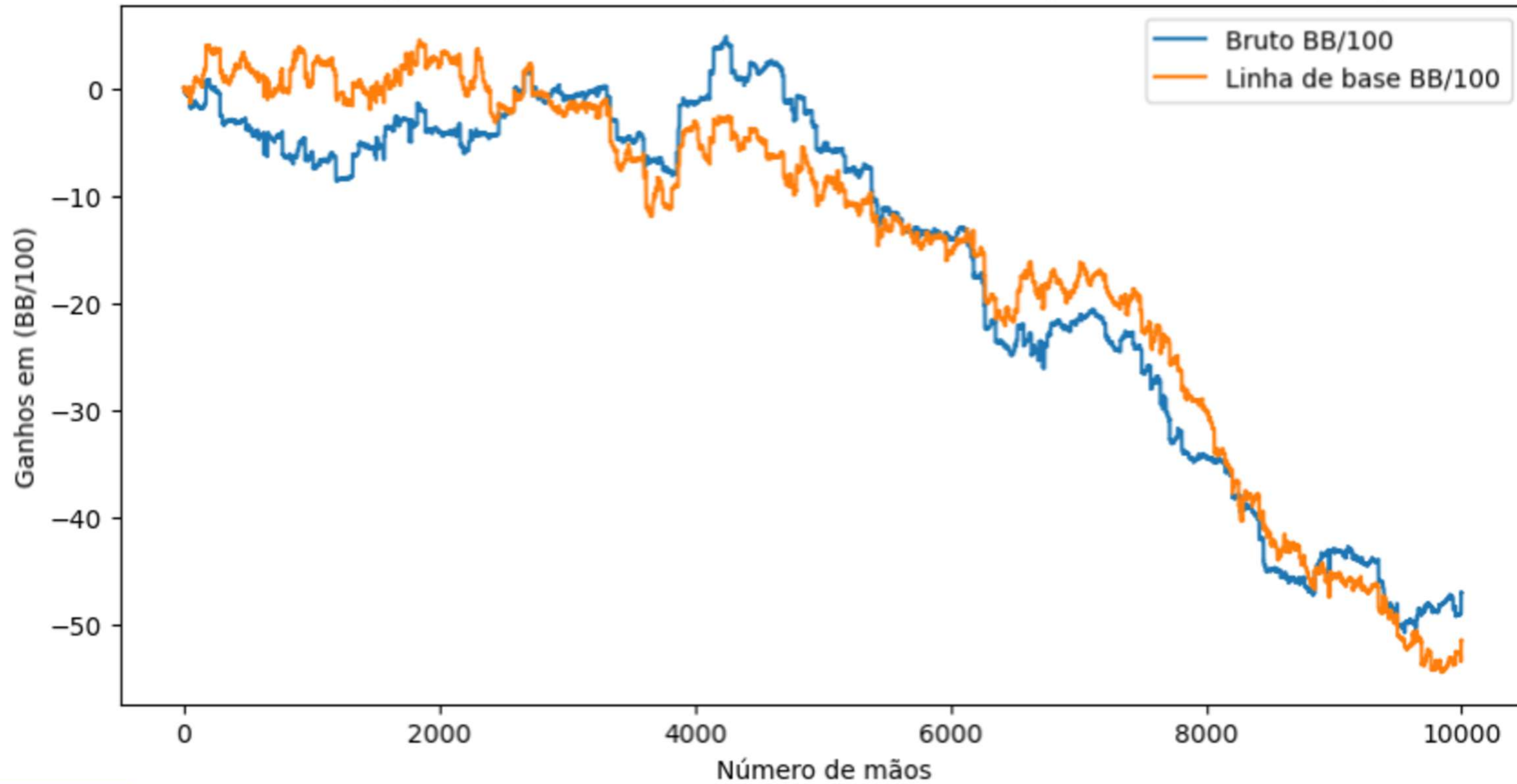


Resultados - CFR com Abstração-Dataset-500.000



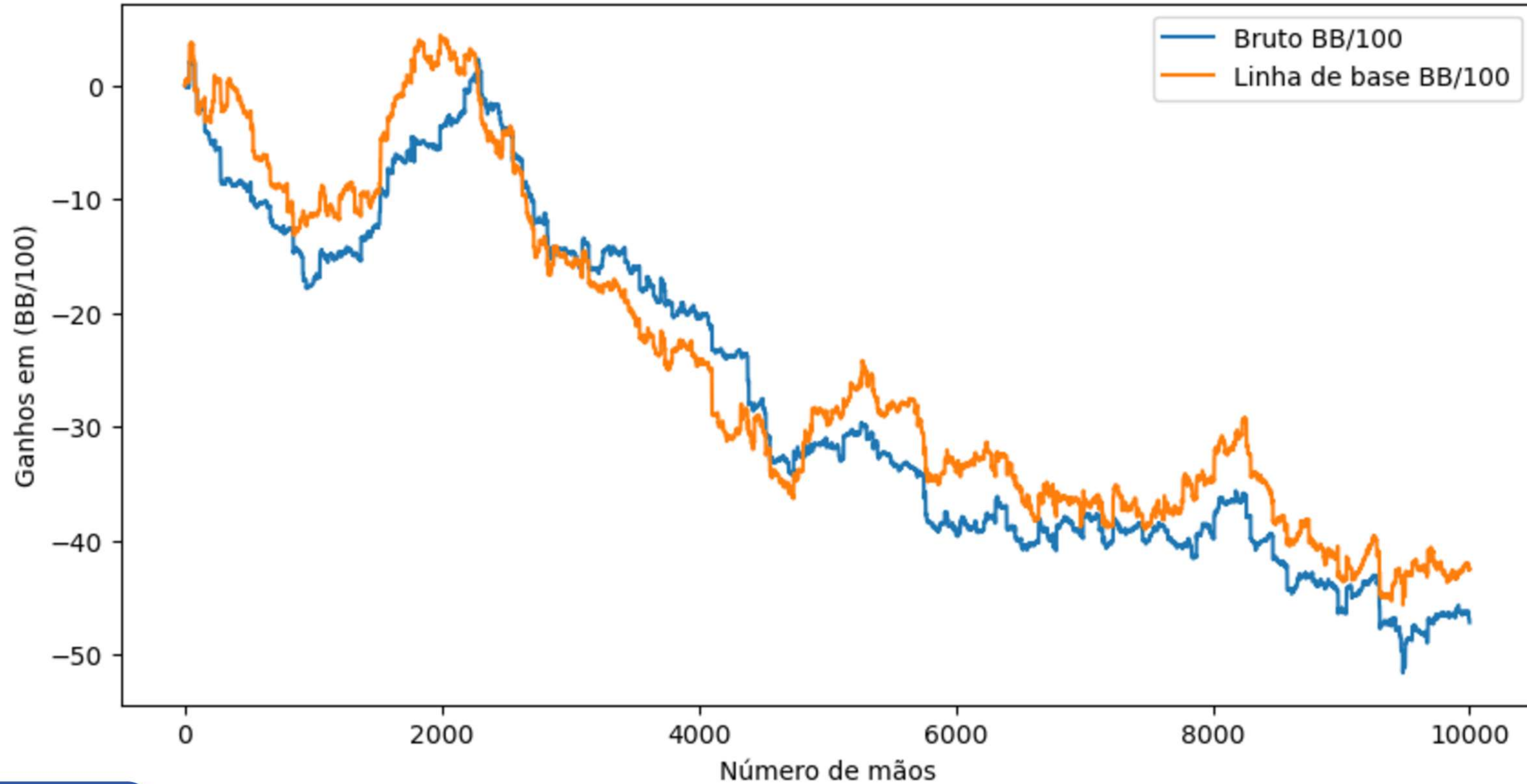
Resultados - CFR com Abstração-Dataset-750.000

Estratégia 4: CFR com abstração Dataset 750.000 (Bruto -47.025 BB/100)



Resultados - CFR com Abstração-Dataset-1.000.000

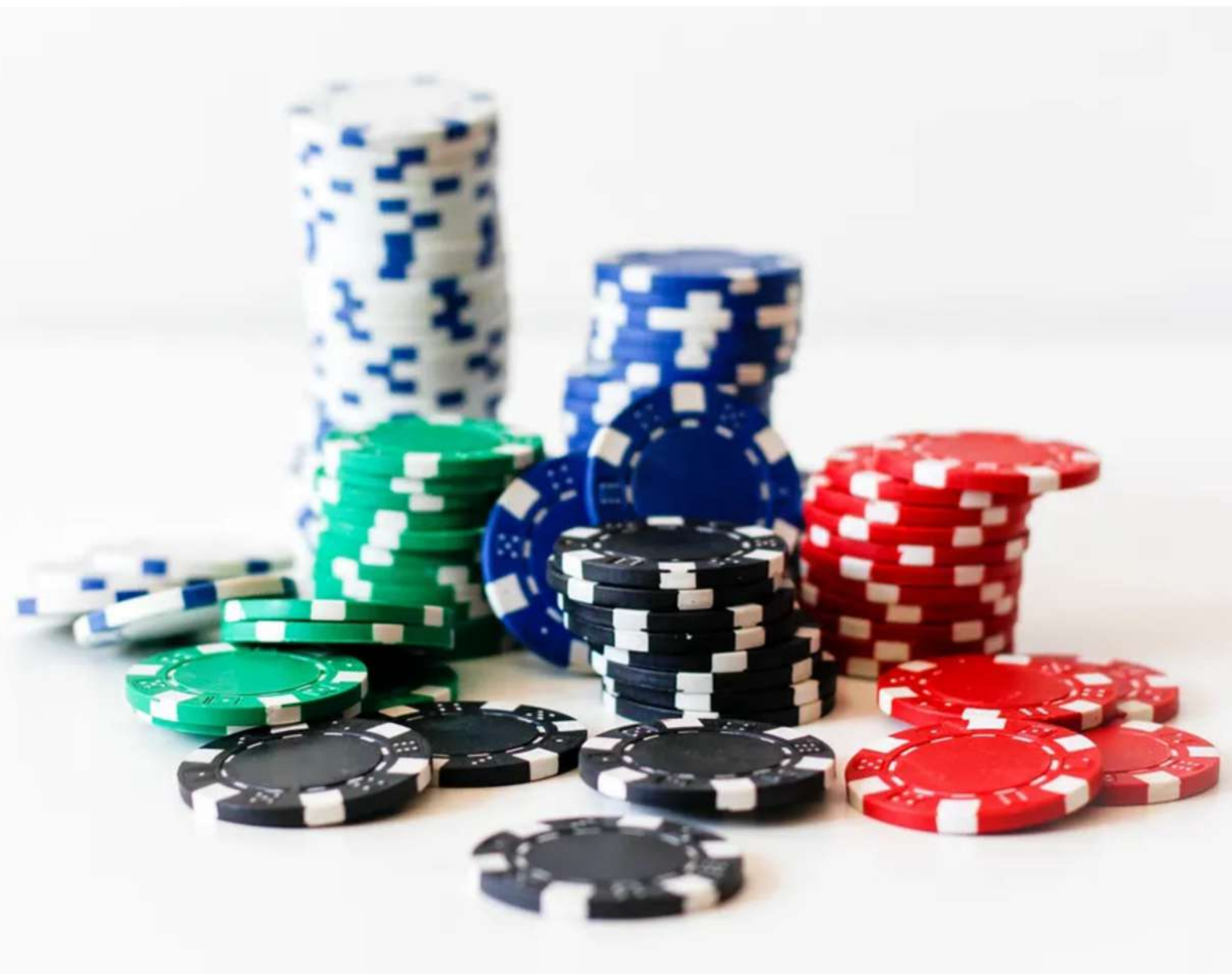
Estratégia 4: CFR com abstração Dataset 1.000.000 (Bruto -47.19 BB/100)



As estratégias heurísticas (0 a 3) apresentaram perdas médias muito altas, variando de -112 a -227 BB/100, confirmando que abordagens baseadas em regras simples são ineficazes contra o Slumbot.

A Estratégia 4, baseada em CFR, mostra uma redução significativa nas perdas médias de (-41,47 a - 47,19 BB/100) em comparação com as heurísticas, indicando que o treinamento com abstração é mais eficaz.

O melhor resultado foi com o *dataset* de 500.000 (-32,125 BB/100), sugerindo um ponto ótimo no tamanho do *dataset* para este contexto.



Conclusão

- Estratégia CFR superou abordagens heurísticas
- Dataset e abstração reduziram a complexidade computacional, viabilizando a execução em hardware modesto.
- Para Futuro: sugere-se explorar algoritmos como CFR+, Monte-Carlo Counterfactual Regret Minimization (MCCFR), além de aprimorar as abstrações.

Obrigado!